

NAME: Muhammad Nayyar Ameer
ROLL# 2530087
PROJECT# CLO4

CLASS:BS_AI
SUBJECT: PRG for AI
DATE: 18th/MAY/2026

SRC CODE:

DENGUE DISEASE DETECTION.csv

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt




# LOAD DATASET


data = pd.read_csv("Dengue diseases dataset.csv")




print(data.head())




print("\n========== DATA INFO ==========")

# ============================================================

# EXPLORATORY DATA ANALYSIS

# ============================================================
```

```
print("\n========== STATISTICAL SUMMARY ==========")
print(data.describe())
print(data.info())



print(data.shape)



print(data.isnull().sum())




print("\n========== DUPLICATE VALUES ==========")
print(data.duplicated().sum())

data = data.drop_duplicates()

# HANDLE MISSING VALUES



# Numerical columns
numerical_cols = data.select_dtypes(include=['int64', 'float64']).columns

# Categorical columns
categorical_cols = data.select_dtypes(include=['object']).columns
```

```python
for col in numerical_cols:
    data[col] = data[col].fillna(data[col].mean())



for col in categorical_cols:
    data[col] = data[col].fillna(data[col].mode()[0])


print("\n========== NULL VALUES AFTER CLEANING ==========")
print(data.isnull().sum())


# ENCODE CATEGORICAL DATA


from sklearn.preprocessing import LabelEncoder


encoder = LabelEncoder()


for col in categorical_cols:
    data[col] = encoder.fit_transform(data[col])


print("\n========== ENCODED DATA ==========")
print(data.head())
```

```
# TARGET CLASS DISTRIBUTION



plt.figure(figsize=(6,5))


data["Final Output"].value_counts().plot(kind='bar')


plt.title("Dengue Detection Distribution")

plt.xlabel("Final Output")

plt.ylabel("Count")


plt.show()


# ==============================

# AGE DISTRIBUTION



plt.figure(figsize=(8,5))


plt.hist(data["Age"], bins=15)


plt.title("Age Distribution")

plt.xlabel("Age")

plt.ylabel("Frequency")


plt.show()
```

```
# ================
#  PLATELET COUNT DISTRIBUTION
# ==================


plt.figure(figsize=(8,5))


plt.hist(data["Platelet Count"], bins=20)


plt.title("Platelet Count Distribution")
plt.xlabel("Platelet Count")
plt.ylabel("Frequency")


plt.show()


# ==================================
# V CORRELATION MATRIX
# ============================


plt.figure(figsize=(10,8))


correlation = data.corr()


plt.imshow(correlation)


plt.colorbar()


plt.xticks(range(len(correlation.columns)), correlation.columns, rotation=90)
```

```
plt.yticks(range(len(correlation.columns)), correlation.columns)


plt.title("Correlation Matrix")


plt.show()


# ============================================================
# FEATURES & TARGET
# ============================================================


x = data.drop("Final Output", axis=1)


y = data["Final Output"]


print("\n========== FEATURES ==========")
print(x.head())


print("\n========== TARGET ==========")
print(y.head())


# ============================================================
# FEATURE SCALING
# ============================================================


from sklearn.preprocessing import StandardScaler


scaler = StandardScaler()
```

```
x_scaled = scaler.fit_transform(x)


#===============
# TRAIN TEST SPLIT



from sklearn.model_selection import train_test_split


x_train, x_test, y_train, y_test = train_test_split(
    x_scaled,
    y,
    test_size=0.2,
    random_state=42
)


print("\n========== TRAIN TEST SHAPE ==========")
print("X Train Shape:", x_train.shape)
print("X Test Shape :", x_test.shape)


# MODEL 1  LOGISTIC REGRESSION



from sklearn.linear_model import LinearRegression


model1 = LinearRegression()
```

```
model1.fit(x_train, y_train)


pred1 = model1.predict(x_test)


# ========================================================================='
# MODEL 2 : DECISION TREE REGRESSOR



from sklearn.tree import DecisionTreeRegressor


model2 = DecisionTreeRegressor()


model2.fit(x_train, y_train)


pred2 = model2.predict(x_test)


# ========================================================
# MODEL 3 RANDOM FOREST REGRESSOR
# =============


from sklearn.ensemble import RandomForestRegressor


model3 = RandomForestRegressor()


model3.fit(x_train, y_train)


pred3 = model3.predict(x_test)
```

```
# MODEL 04:: KNN
from sklearn.neighbors import KNeighborsRegressor

model4 = KNeighborsRegressor()

model4.fit(x_train, y_train)

pred4 = model4.predict(x_test)
# ============================================================
# EVALUATION

from sklearn.metrics import r2_score



# Logistic Regression
acc1 = r2_score(y_test, pred1)



# Decision Tree
acc2 = r2_score(y_test, pred2)



# Random Forest
acc3 = r2_score(y_test, pred3)

#KNN
```

```
acc4 = r2_score(y_test, pred4)



# ================================
# RESULTS



print(" MODEL COMPARISOn")


print("Logistic Regression")
print("Accuracy :", acc1)



print(" Decision Tree")
print("Accuracy :", acc2)



print(" Random Forest")
print("Accuracy :", acc3)

print("KNN")
print("Accuracy :", acc4)



# ==============
# SAMPLE :: PREDICTION
```

```
sample = [[
    35,     # Age
    1,      # Sex
    12.5,   # Haemoglobin
    3000,   # WBC Count
    1,      # Differential Count
    1,      # RBC PANEL
    40000,  # Platelet Count
    15.0    # PDW
]]

sample = scaler.transform(sample)

prediction = model3.predict(sample)

print("\n========== SAMPLE PREDICTION ==========")

if prediction[0] == 1:
    print("Dengue Detected")
else:
    print("No Dengue Detected")

# ========================================================
# BEST MODEL
# ========================================================

best_accuracy = max(acc1, acc2, acc3,acc4)
```

```
print("====== BEST MODEL ==========")

if best_accuracy == acc1:
    print("Logistic Regression performed best.")

elif best_accuracy == acc2:
    print("Decision Tree performed best.")
elif best_accuracy == acc4:
    print("KNN performed best.")
else:
    print("Random Forest performed best.")

# ============================================================
# FINAL ANALYSIS
# ============================================================

print("\n========== PROJECT ANALYSIS ==========")

print("""
1. Dataset was loaded and cleaned properly.
2. Missing values were handled using mean and mode methods.
3. Categorical features were encoded using Label Encoding.
4. Feature scaling was applied using StandardScaler.
5. Multiple supervised learning models were trained.
6. Random Forest generally performs best for medical datasets.
7. Visualizations helped understand feature distributions and relationships.
8. Platelet Count and WBC Count are important indicators for dengue detection.
```

```
9. The model can help in early dengue disease prediction.
""")
```

OUTPUT:

```
  Age     Sex  Haemoglobin  WBC Count  Differential Count  RBC PANEL
\
0   43    Male         12.6     2200.0                   1          1
1   45    Male         13.2     3000.0                   0          1
2   50  Female         11.0     3300.0                   1          1
3   57  Female         11.9     3500.0                   1          0
4   51  Female         13.0     3100.0                   0          1

   Platelet Count   PDW  Final Output
0         62000.0  11.0           1.0
1         17000.0  17.0           1.0
2         19000.0  16.3           1.0
3         29000.0  14.0           1.0
4         30000.0  14.5           1.0

========== DATA INFO ==========

========== STATISTICAL SUMMARY ==========
               Age  Haemoglobin     WBC Count  Differential Count  \
count  1003.000000  1003.000000    979.000000         1003.000000
mean     42.252243    13.704985   4319.816139            0.938185
std      20.973160     1.482410   2333.300226            0.240939
min       3.000000    11.000000   2000.000000            0.000000
25%      27.000000    12.600000   2600.000000            1.000000
```

```
50%      41.000000    13.700000   3200.000000            1.000000
75%      55.000000    15.000000   5900.000000            1.000000
max     120.000000    25.000000  10900.000000            1.000000

         RBC PANEL  Platelet Count         PDW  Final Output
count  1003.000000      986.000000  984.000000    989.000000
mean      0.937188   114202.719067   22.741768      0.676441
std       0.242745    89000.982480   14.619862      0.468071
min       0.000000    10000.000000    1.000000      0.000000
25%       1.000000    46000.000000   14.000000      0.000000
50%       1.000000    92000.000000   17.700000      1.000000
75%       1.000000   162500.000000   27.800000      1.000000
max       1.000000   500000.000000  215.000000      1.000000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1003 entries, 0 to 1002
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Age                 1003 non-null   int64
 1   Sex                 1003 non-null   object
 2   Haemoglobin         1003 non-null   float64
 3   WBC Count           979 non-null    float64
 4   Differential Count  1003 non-null   int64
 5   RBC PANEL           1003 non-null   int64
 6   Platelet Count      986 non-null    float64
 7   PDW                 984 non-null    float64
 8   Final Output        989 non-null    float64
dtypes: float64(5), int64(3), object(1)
memory usage: 70.7+ KB
None
(1003, 9)
Age                    0
Sex                    0
Haemoglobin            0
WBC Count             24
Differential Count     0
RBC PANEL              0
Platelet Count        17
PDW                   19
```

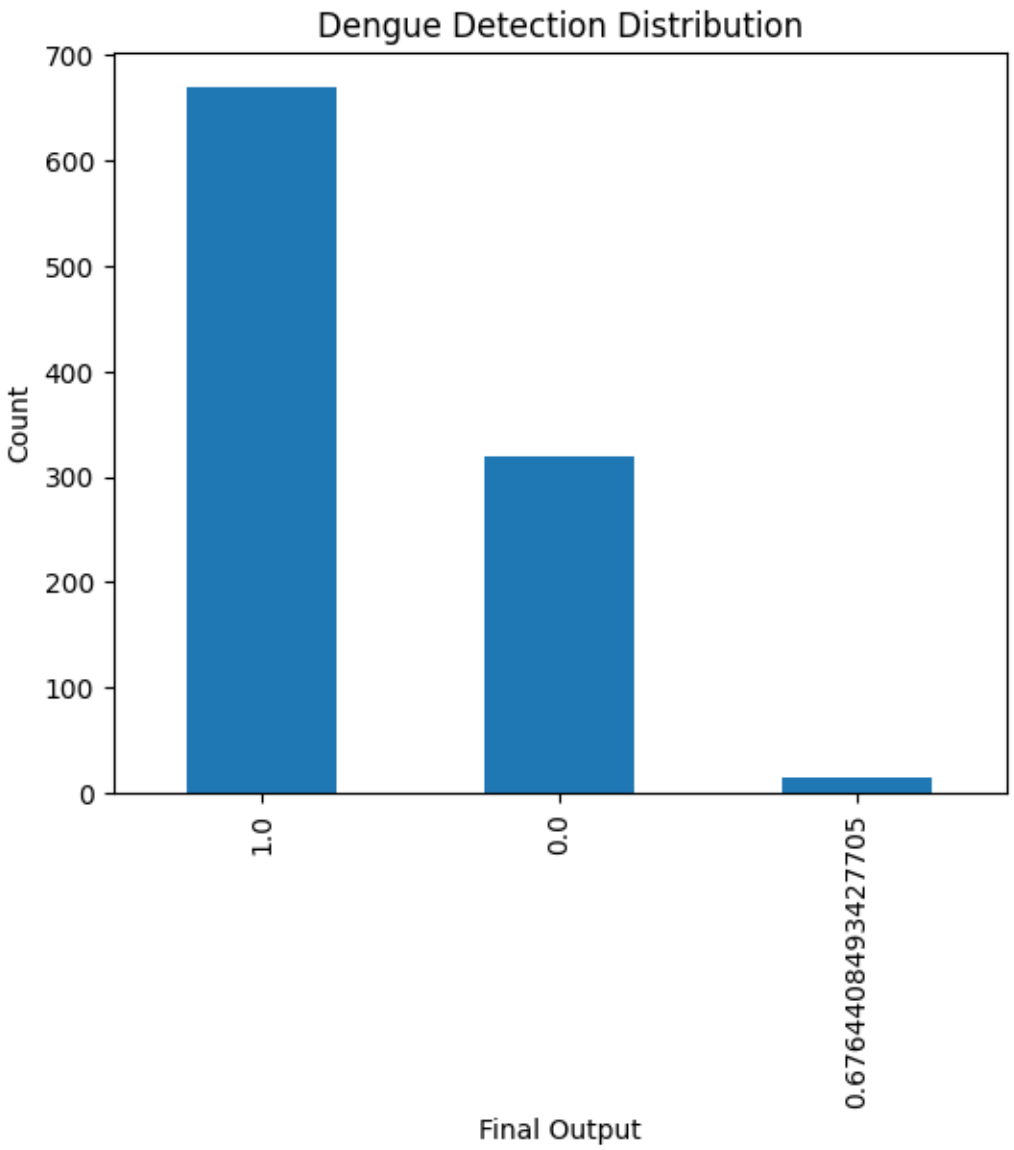
```
Final Output          14
dtype: int64

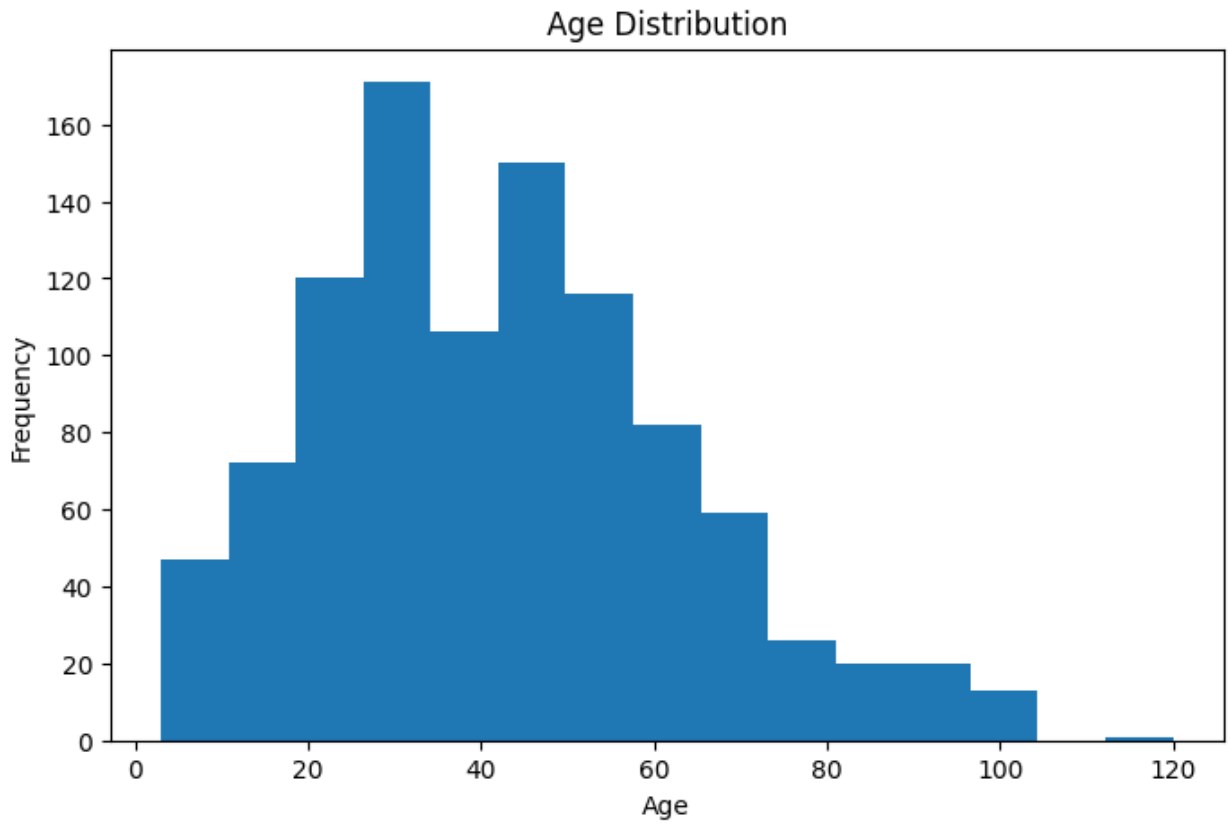
========== DUPLICATE VALUES ==========
0

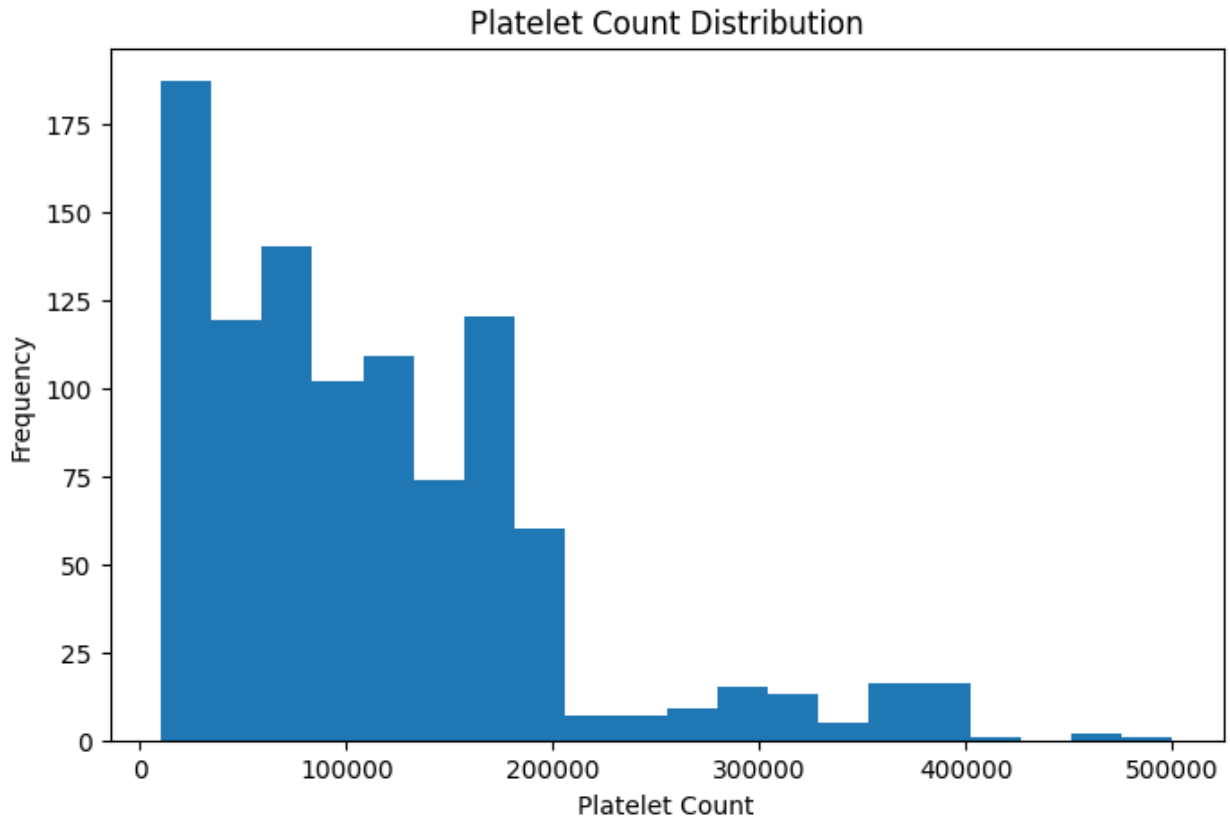
========== NULL VALUES AFTER CLEANING ==========
Age                   0
Sex                   0
Haemoglobin           0
WBC Count             0
Differential Count    0
RBC PANEL             0
Platelet Count        0
PDW                   0
Final Output          0
dtype: int64

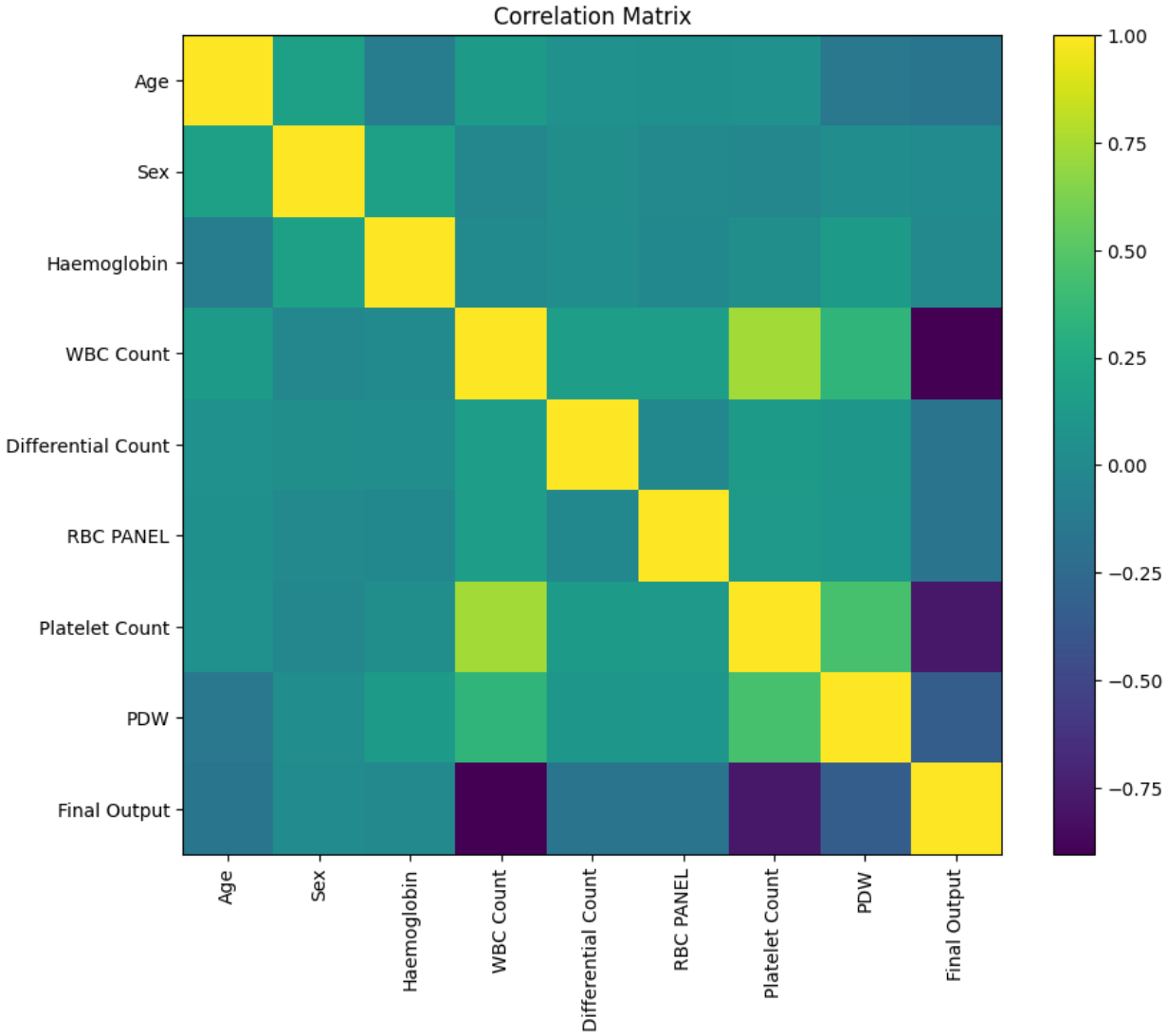
========== ENCODED DATA ==========
   Age  Sex  Haemoglobin  WBC Count  Differential Count  RBC PANEL  \
0   43    2         12.6     2200.0                   1          1
1   45    2         13.2     3000.0                   0          1
2   50    1         11.0     3300.0                   1          1
3   57    1         11.9     3500.0                   1          0
4   51    1         13.0     3100.0                   0          1

   Platelet Count   PDW  Final Output
0         62000.0  11.0           1.0
1         17000.0  17.0           1.0
2         19000.0  16.3           1.0
3         29000.0  14.0           1.0
4         30000.0  14.5           1.0
```


Dengue Detection Distribution
700
600
500
400
300
200
100
0
Count
1.0
0.0
0.6764408493427705
Final Output


Age Distribution
Frequency
Age
0
20
40
60
80
100
120
0
20
40
60
80
100
120
140
160


Platelet Count Distribution
Frequency
Platelet Count
0
100000
200000
300000
400000
500000
0
25
50
75
100
125
150
175




```
========== FEATURES ==========
   Age  Sex  Haemoglobin  WBC Count  Differential Count  RBC PANEL  \
0   43    2         12.6     2200.0                   1          1
1   45    2         13.2     3000.0                   0          1
2   50    1         11.0     3300.0                   1          1
3   57    1         11.9     3500.0                   1          0
4   51    1         13.0     3100.0                   0          1

   Platelet Count   PDW
0         62000.0  11.0
1         17000.0  17.0
2         19000.0  16.3
3         29000.0  14.0
```

```
4         30000.0  14.5

========== TARGET ==========
0    1.0
1    1.0
2    1.0
3    1.0
4    1.0
Name: Final Output, dtype: float64

========== TRAIN TEST SHAPE ==========
X Train Shape: (802, 8)
X Test Shape : (201, 8)
 MODEL COMPARISOn
Logistic Regression
Accuracy : 0.8355693893241751
 Decision Tree
Accuracy : 0.9397768517081262
 Random Forest
Accuracy : 0.9586438870903201
KNN
Accuracy : 0.9437162119176016

========== SAMPLE PREDICTION ==========
Dengue Detected
====== BEST MODEL ==========
Random Forest performed best.

========== PROJECT ANALYSIS ==========

1. Dataset was loaded and cleaned properly.
2. Missing values were handled using mean and mode methods.
3. Categorical features were encoded using Label Encoding.
4. Feature scaling was applied using StandardScaler.
5. Multiple supervised learning models were trained.
6. Random Forest generally performs best for medical datasets.
7. Visualizations helped understand feature distributions and
relationships.
8. Platelet Count and WBC Count are important indicators for dengue
```

detection.
9. The model can help in early dengue disease prediction.